

Problem Set 5
COMP301 FALL 2025
Week 6: 17.03.2025 - 21.03.2025

Instructions:

- Submit your answers to the Learnhub PS5 assignment until March 22 Saturday, at 23.59.
- Please submit only **one single PDF file**, where all of your written solutions or codes and outputs for each of the parts are included.
- Name your submission file as *id_username_ps5.pdf*
(Example: *00000_raliyev22_ps5.pdf*).

Problem 1: In the lecture you have seen a procedural implementation of the environment. Below, there is a procedural implementation of the list. Fill in the blank part.

```
(define (empty-list)
  (lambda (mode)
    (display "end_of_list")))

(define (prepend-list a lst)
  (lambda (mode)
    (-----
     -----
     -----)))

(define (car-list lst)
  (lst #t))

(define (cdr-list lst)
  (lst #f))

; Tests:
(define x
  (prepend-list 13
    (prepend-list 3
      (prepend-list 6
        (prepend-list 7
          (empty-list))))))

(car-list x) ; returns 13
(car-list (cdr-list x)) ; returns 3
(car-list
  (cdr-list
    (cdr-list
      (cdr-list (cdr-list x))))) ; returns "end of list"
```

Problem 2: You have previously seen a procedural implementation of the list. In racket, it is also possible to implement struct-like data types using procedures. Below, you can find a procedural implementation of the point. Point is a data type that keeps the three coordinates x, y, and z. You are expected to implement the getters, setters and get-distance methods by completing the indicated sections.

Hint: To have a better understanding of how set! method works with procedure calls you can play with the following code fragment and observe the output:

```
(define (counter tic)
  (lambda ()
    (set! tic (+ tic 1)) tic))

; run it as following:
; (define count (counter 0))
; (count)
; (count) ; See what happens? ;
; -----END OF HINT----- ;

(define (create-point x y z)
  (lambda (mode)
    (cond
      ((= mode 0) x)
      ((= mode 1)
       ; Your code here ;

      )
      ((= mode 2)
       ; your code here ;

      )
      ((= mode 3)
       (lambda (new-x)
         ; Your code here ;
         ; Hint: You can use the set! function to change the value of a variable ;
       ))
      ((= mode 4)
       ; Your code here ;

      )
      ((= mode 5)
       ; Your code here ;

      )))

(define (get-x point)
  (point 0))

(define (get-y point)
  ; Your code here ;
)

(define (get-z point)
  ; Your code here ;
)

(define (set-x point new-x)
  ((point 3) new-x))
```

```
(define (set-y point new-y)
  ((point 4) new-y))
```

```
(define (set-z point new-z)
  ((point 5) new-z))
```

```
(define (square x)
  (* x x))
```

```
(define (get-distance point-1 point-2)
  ; Your code here ;

)
```

```
; Tests:
; (get-distance (create-point 3 5 9) (create-point 5 3 8)) ; Should return 3 ;
; (define p (create-point 0 0 0))
; (get-x p) ; Should return 0 ;
; (set-x p 5) ; Returns nothing ;
; (get-x p) ; Should now return 5 ;list"
```

Question 1

```
(define (prepend-list a lst)
  (lambda (mode)
    (if mode a lst)
  ))
```

Question 2

```
(define (create-point x y z)
  (lambda (mode)
    (cond
      ((= mode 0) x)
      ((= mode 1)
       y
      )
      ((= mode 2)
       z
      )
      ((= mode 3)
       (lambda (new-x)
         (set! x new-x)
       ))
      ((= mode 4)
       (lambda (new-y)
         (set! y new-y)
       ))
      ((= mode 5)
       (lambda (new-z)
         (set! z new-z)
       ))
    ))))
```

```
(define (get-x point)
  (point 0))
(define (get-y point)
  (point 1)
)
(define (get-z point)
  (point 2)
)
(define (set-x point new-x)
  ((point 3) new-x))
(define (set-y point new-y)
  ((point 4) new-y))
(define (set-z point new-z)
  ((point 5) new-z))
(define (square x)
  (* x x))
(define (get-distance point-1 point-2)
  (sqrt (+ (square (- (get-x point-1) (get-x point-2))) (square (- (get-y point-1) (get-y point-2))) (square (- (get-z point-1) (get-z point-2))))))
)
```